Lightweight Convolutional Neural Network

Dr. Oybek Eraliev,

Department of Computer Engineering

Inha University In Tashkent.

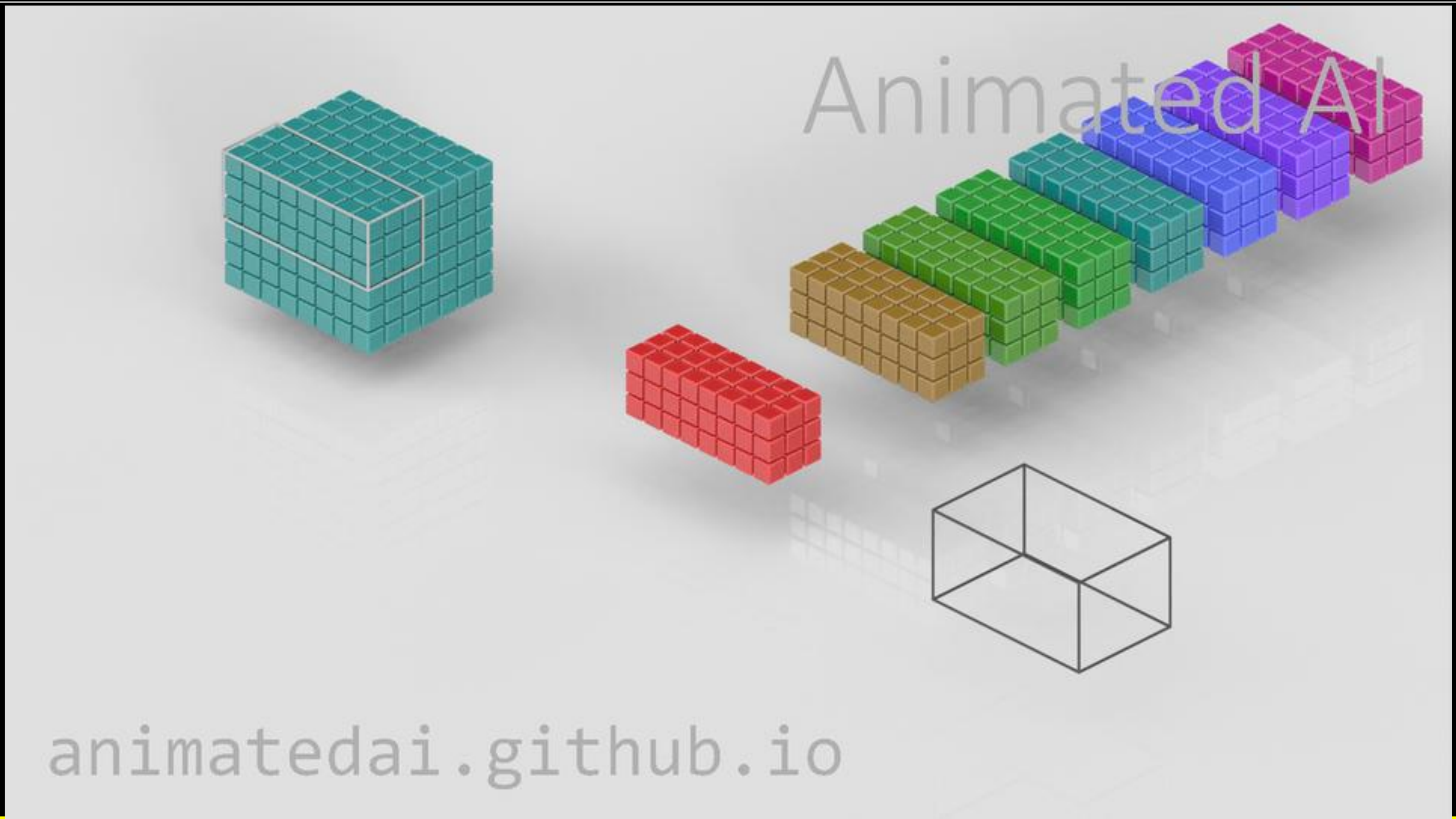
Email: oybekeraliev7@gmail.com

Content

- Standard Conv Layer
- Grouped Conv Layer
- Depthwise Separable Conv Layer
- BatchNorm Layer
- Implementation of CNN using PyTorch Framework

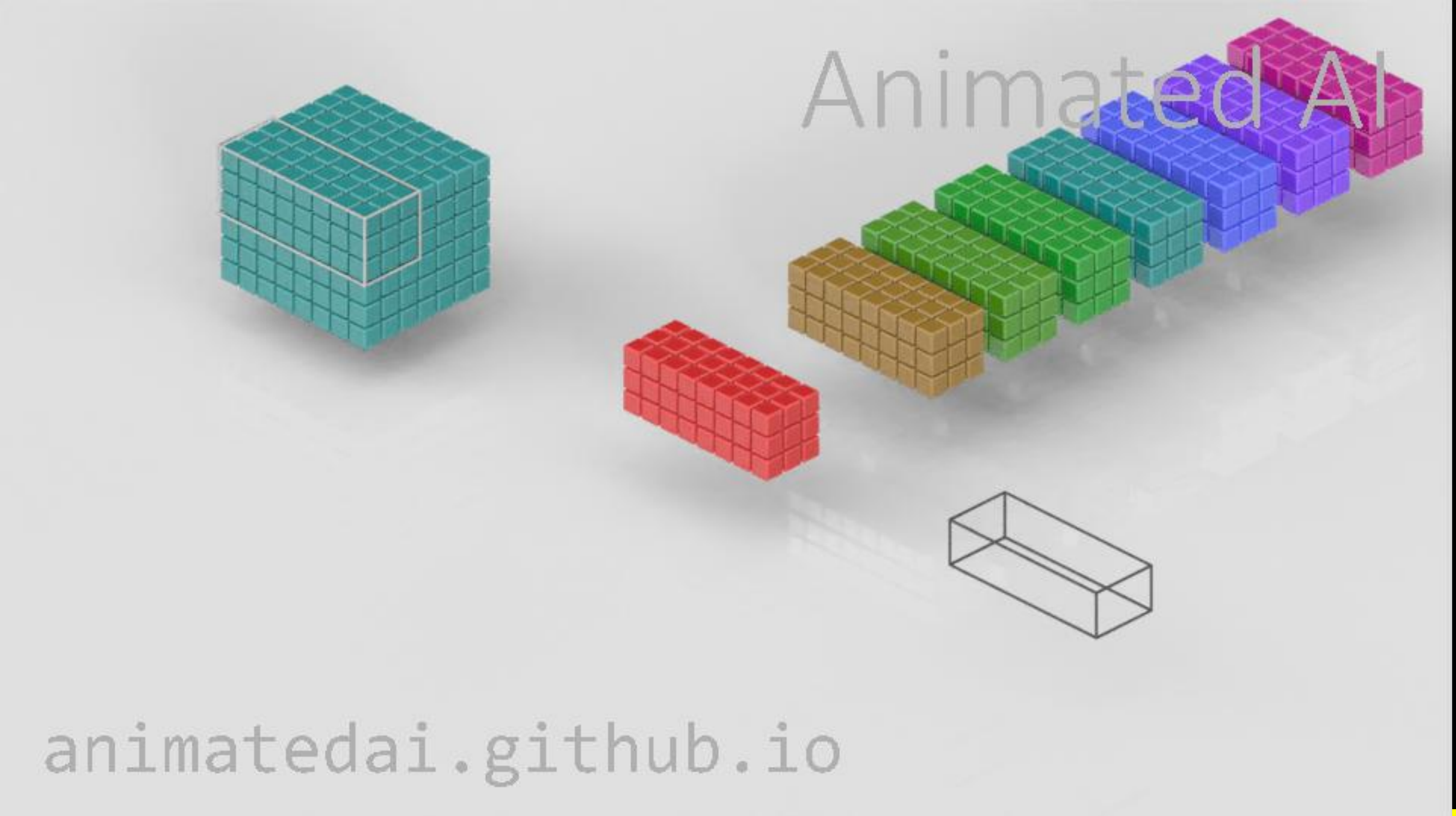
Standard Conv Layer

3×3 filter, Stride = 1, Padding = 0



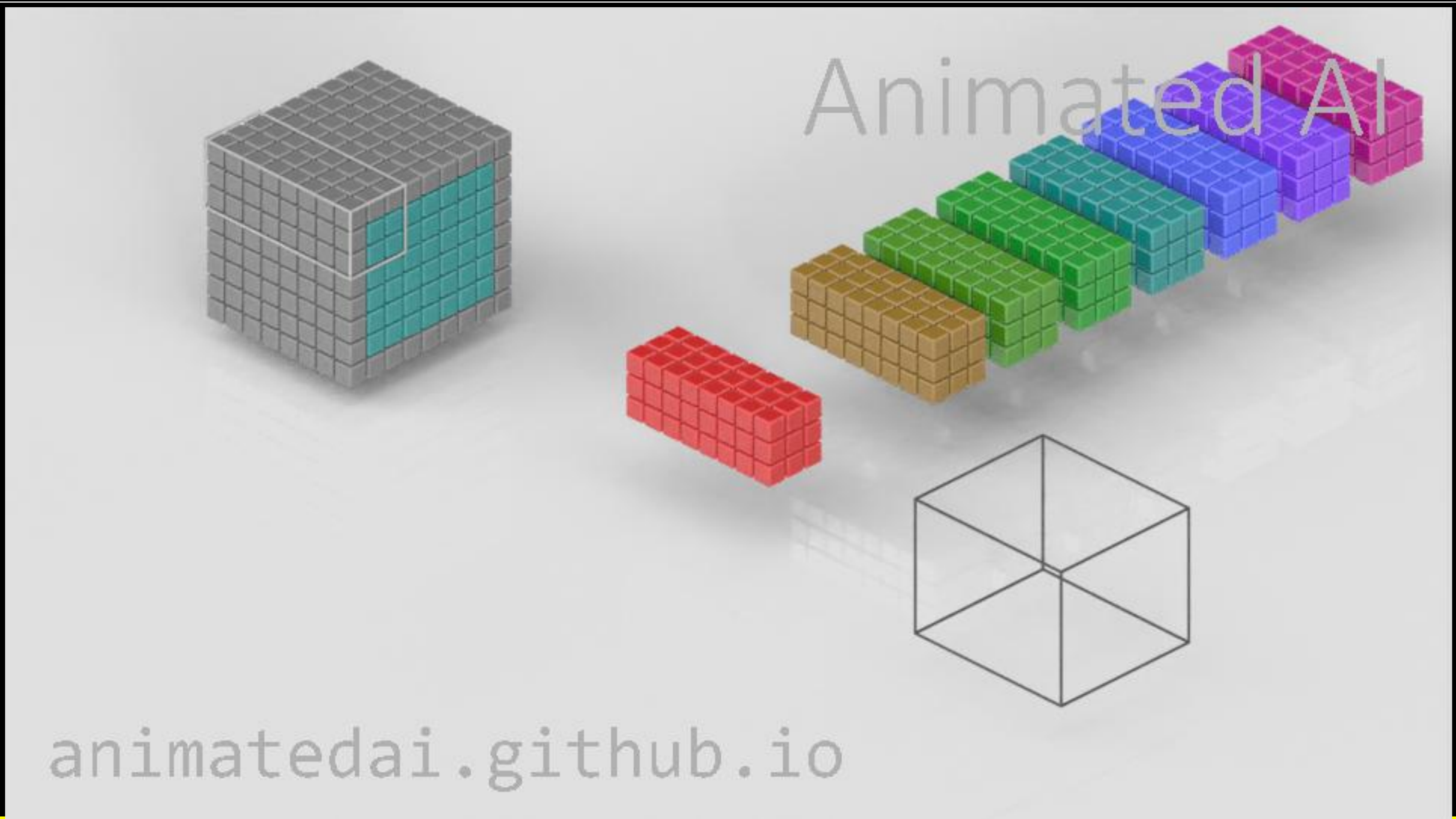
Standard Conv Layer

3×3 filter, Stride $= 2$, Padding $= 0$



Standard Conv Layer

3×3 filter, Stride $= 1$, Padding $= 1$

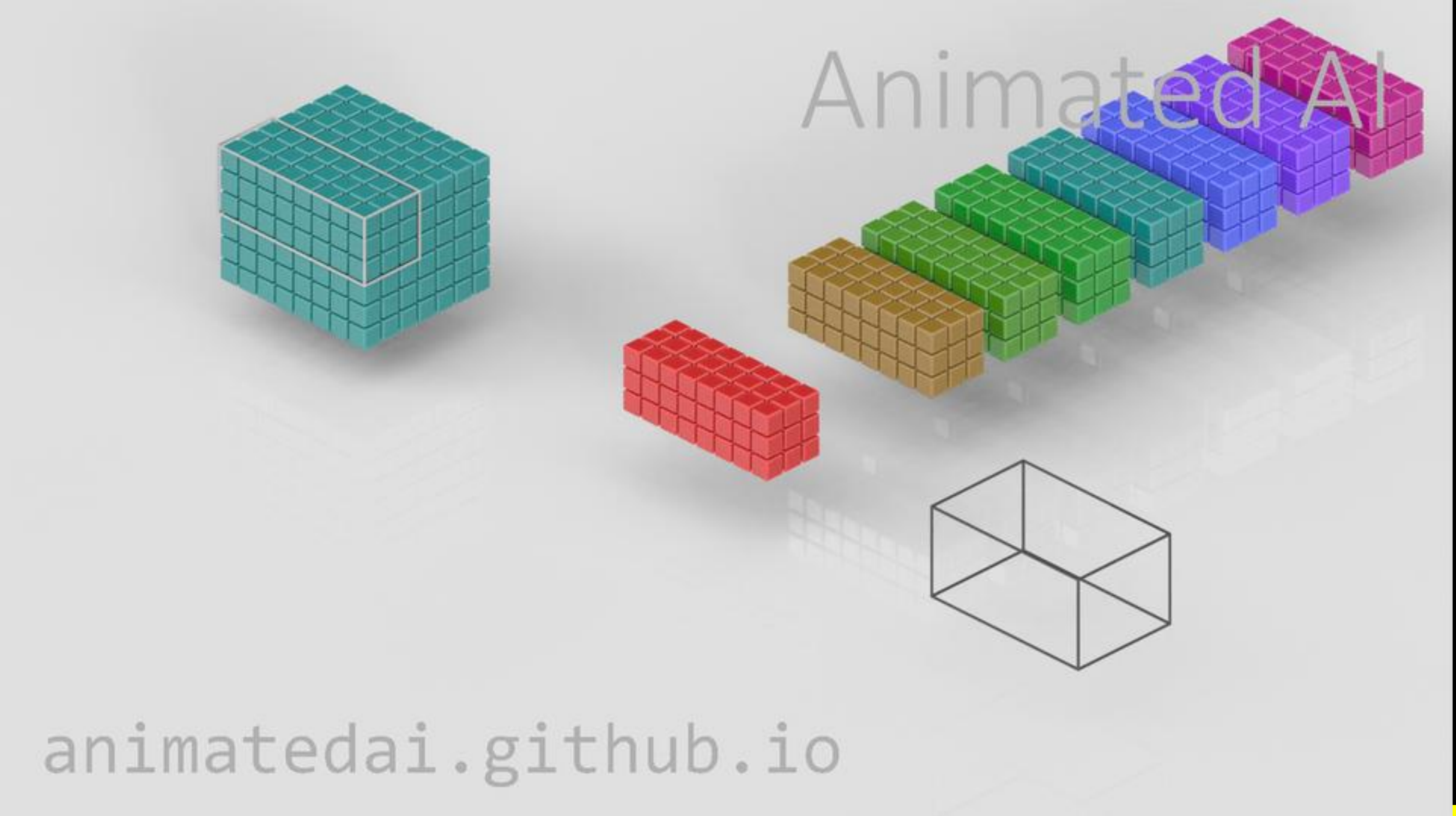


Content

- ~~Standard Conv Layer~~
- Grouped Conv Layer
- Depthwise Separable Conv Layer
- BatchNorm Layer
- Implementation of CNN using PyTorch Framework

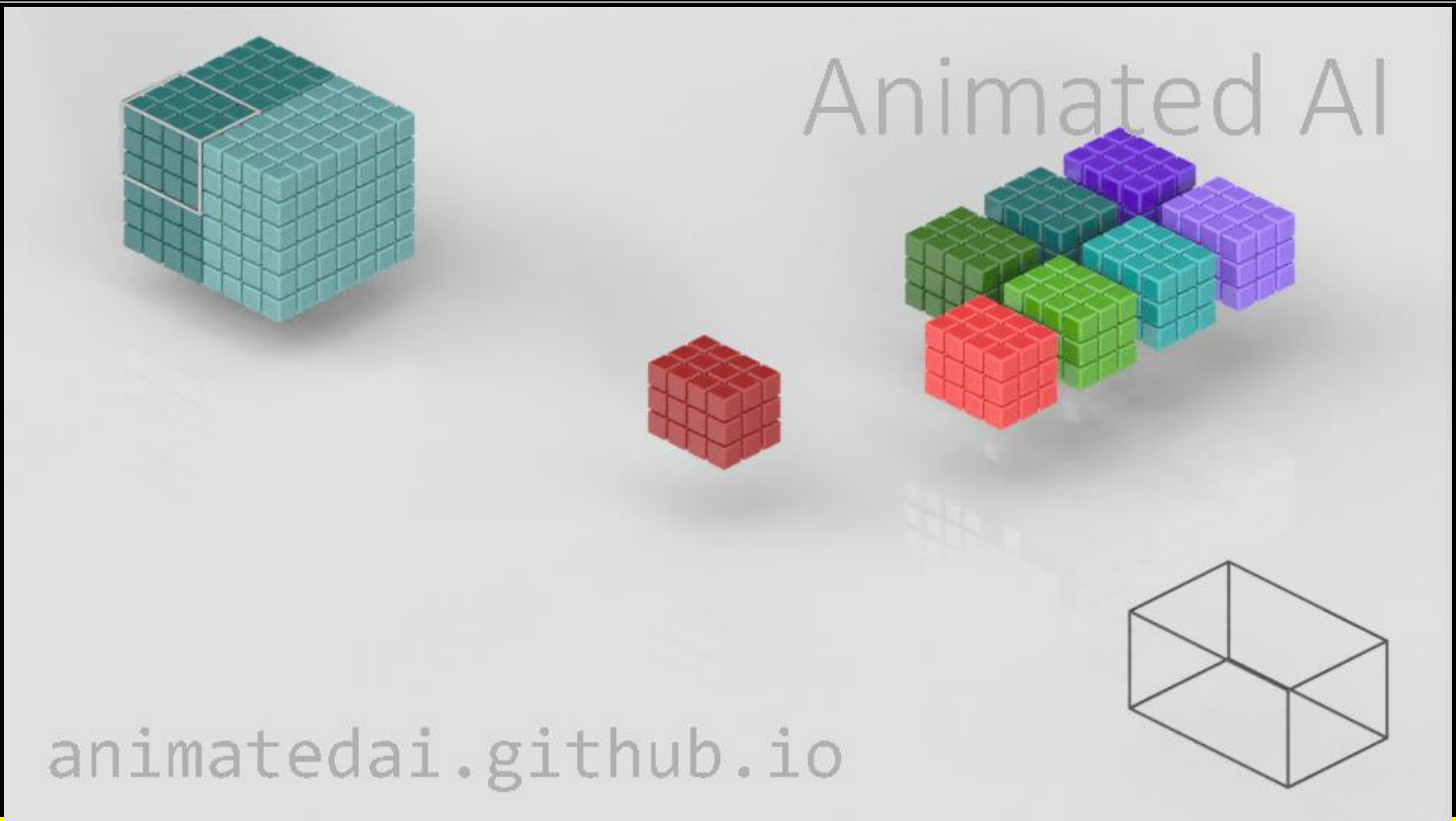
Grouped Conv Layer

3×3 filter, Stride $= 1$, Padding $= 0$, groups $= 1$



Grouped Conv Layer

3×3 filter, Stride $= 1$, Padding $= 0$, groups $= 2$

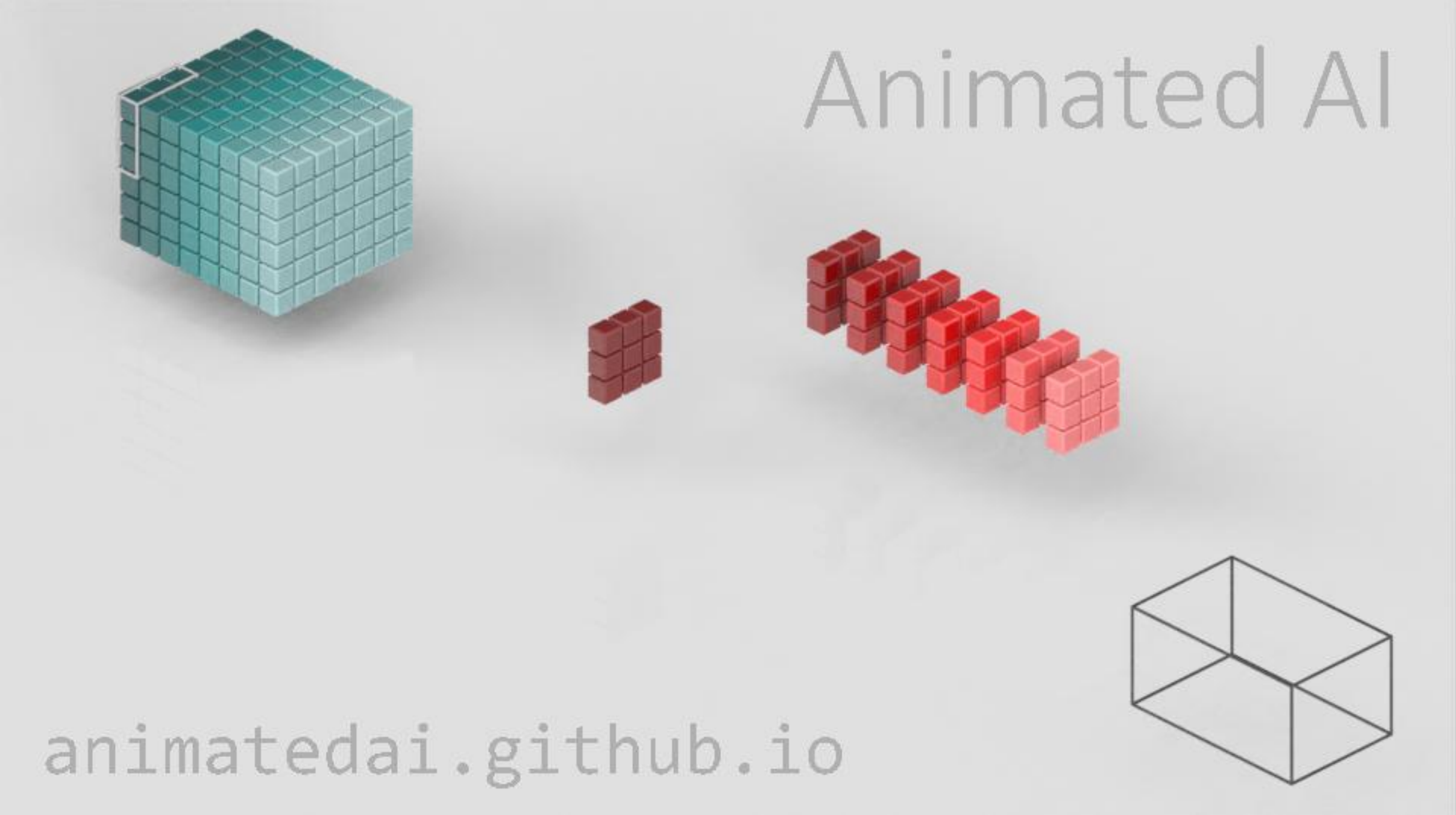


Content

- ~~Standard Conv Layer~~
- ~~Grouped Conv Layer~~
- Depthwise Separable Conv Layer
- BatchNorm Layer
- Implementation of CNN using PyTorch Framework

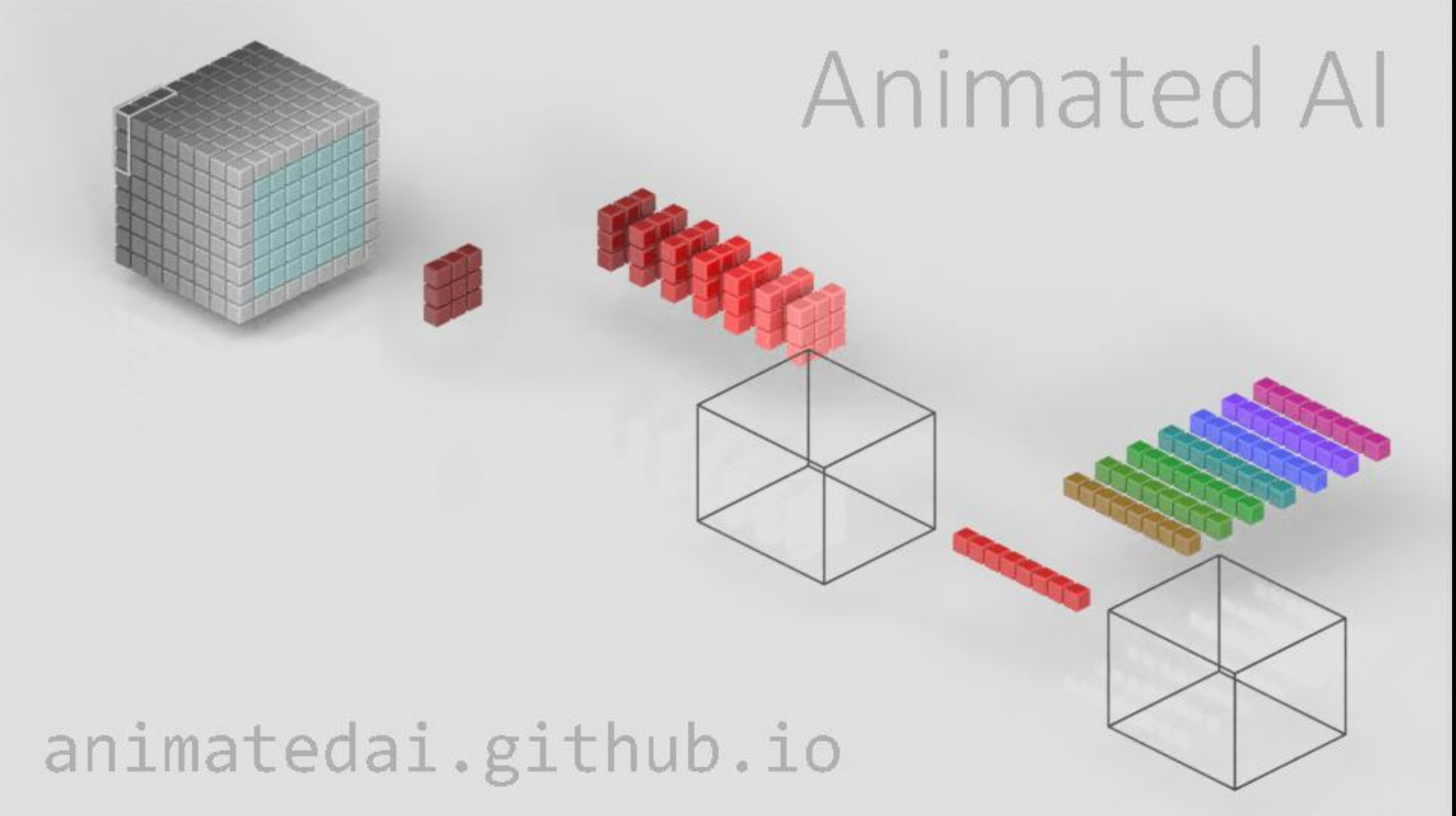
Depthwise Separable Conv Layer

3×3 filter, Stride $= 1$, Padding $= 0$, groups $= 8$



Depthwise Separable Conv Layer

3×3 filter, Stride = 1, Padding = 0, groups = 8



Content

- ~~Standard Conv Layer~~
- ~~Grouped Conv Layer~~
- ~~Depthwise Separable Conv Layer~~
- BatchNorm Layer
- Implementation of CNN using PyTorch Framework

BatchNorm Layer

As the name suggests, batch normalization is a technique where batched training data, after activation in the current layer and before moving to the next layer, is standardized. Here's how it works:

1. *The entire dataset is randomly divided into N batches without replacement, each with a mini_batch size, for the training.*
2. *For the i-th batch, standardize the data distribution within the batch using the formula: (Xi - Xmean) / Xstd.*
3. *Scale and shift the standardized data with $\gamma Xi + \beta$ to allow the neural network to undo the effects of standardization if needed.*


THANK YOU